

ThreMoLIA: Threat Modeling of Large Language Model-Integrated Applications

Felix Viktor Jedrzejewski
Blekinge Institute of Technology
Karlskrona, Sweden
felix.jedrzejewski@bth.se

Davide Fucci
Blekinge Institute of Technology
Karlskrona, Sweden
davide.fucci@bth.se

Oleksandr Adamov
Blekinge Institute of Technology
Karlskrona, Sweden
oleksandr.adamov@bth.se

Abstract

Large Language Models (LLMs) are currently being integrated into industrial software applications to help users perform more complex tasks in less time. However, these LLM-Integrated Applications (LIA) expand the attack surface and introduce new kinds of threats. Threat modeling is commonly used to identify these threats and suggest mitigations. However, it is a time-consuming practice that requires the involvement of a security practitioner. Our goals are to 1) provide a method for performing threat modeling for LIAs early in their lifecycle, (2) develop a threat modeling tool that integrates existing threat models, and (3) ensure high-quality threat modeling. To achieve the goals, we work in collaboration with our industry partner. Our proposed way of performing threat modeling will benefit industry by requiring fewer security experts' participation and reducing the time spent on this activity. Our proposed tool combines LLMs and Retrieval Augmented Generation (RAG) and uses sources such as existing threat models and application architecture repositories to continuously create and update threat models. We propose to evaluate the tool offline—i.e., using benchmarking—and online with practitioners in the field. We conducted an early evaluation using ChatGPT on a simple LIA and obtained results that encouraged us to proceed with our research efforts.

CCS Concepts

• Security and privacy → Software security engineering; • Software and its engineering → Designing software.

Keywords

Threat Modeling, LLM-integrated Applications, Secure Software Engineering, AI4SE, and SE4AI.

ACM Reference Format:

Felix Viktor Jedrzejewski, Davide Fucci, and Oleksandr Adamov. 2025. ThreMoLIA: Threat Modeling of Large Language Model-Integrated Applications. In *Evaluation and Assessment in Software Engineering (EASE '25)*, June 17–20, 2025, Istanbul, Turkiye. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3756681.3757083>

1 Introduction

Large Language Models (LLM) are being integrated into traditional software to enhance their capabilities and performance [9].

However, LLMs extend the threat landscape of an application as attackers can exploit new types of vulnerability [12], such as prompt injections [7]. Prompt injections can force a model to generate malicious outputs, but carefully crafted instructions—introduced during the post-training phase as guardrails—help mitigate this threat [4, 6, 27]. In the early stages of LLM-Integrated Application (LIA) development, the process of threat modeling facilitates the systematic identification of threats, their risk, as well as corresponding mitigations and their priorities [13, 17, 24]. Threat modeling also facilitates communicating security risks between stakeholders (e.g., security experts, developers, architects) [31]. A *threat model* is the result of a threat modeling process; it lists the identified threats and corresponding mitigation strategies for a given application [29].

However, current threat modeling tools (i.e., Microsoft Threat Modeling Tool, OWASP Threat Dragon, and ThreatModeler) are i) lacking maturity as they require manual efforts and security-related skills [31] and ii) not adapted to LLM-specific threats. LIAs contain dynamic and nondeterministic LLM components, which necessitate a new or adapted threat modeling method to identify and mitigate LLM-specific threats. While the industrial maturity and tool support for threat modeling is low [31], the threat models proposed in academia are unsuitable for industrial purposes [8]. Practitioners in industries such as telecommunication are recognizing the potential of Artificial Intelligence (AI) and plan to incorporate it in their systems [1] despite threat modeling methods for LIAs not being validated empirically in industrial contexts.

In this paper, we present our vision of an approach assisting practitioners during the threat modeling of LIAs based on LLM.

Automating threat modeling with an LLM enables security practitioners to accelerate development while maintaining an up-to-date threat model throughout the development lifecycle. Our approach, developed with an industry partner, prioritizes the quality of threat modeling reports by implementing comprehensive data checks to ensure the accuracy and reliability of the output. To the best of our knowledge, this paper is the first to report an LLM-supported threat modeling approach for LIAs co-produced with industry.

The rest of this paper is structured as follows. In Section 2, we summarize the research efforts in the area of threat modeling for LLM and describe the components contributing to our adopted threat modeling method for LIAs. In Section 3, we describe our vision to develop and validate our threat modeling approach in collaboration with our industry partner, including foreseen challenges. Section 4 summarizes our preliminary results. Section 5 concludes the paper.



This work is licensed under a Creative Commons Attribution 4.0 International License. EASE '25, Istanbul, Turkiye
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1385-9/25/06
<https://doi.org/10.1145/3756681.3757083>

2 Background

This section elaborates on the general concept of threat modeling.

2.1 Threat Modeling

The goal of every threat modeling is “(...) to create an abstraction of the system; profiles of potential attackers, including their goals and methods; and a catalog of potential threats that may arise.” [20]. Performing threat modeling in an early stage of software development (i.e., architecture and design) provides security practitioners with possibilities to identify and mitigate threats. Architects and developers then use these mitigations to improve the software design [20]. Several threat modeling methods exist whose application is context-dependent. [20].

According to the well-established practice from Microsoft and OWASP, threat modeling methods use Data-Flow Diagrams (DFDs) to visually represent the system under review by “(...) offering a high-level yet detailed representation of applications’ architecture(...) and its internal and external data flows” [19]. There are a variety of DFD styles, but all of them share the same four base item groups—*external entities*, *data flows*, *processes*, and *data stores* [19]. At the moment, DFDs are the only type of diagram we consider for visualizing the system under review.

STRIDE—i.e., Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege—is a prominent threat modeling framework to identify and classify threats in a given software system [16]. It organizes possible threats across five categories—spoofing, tampering, repudiation, information disclosure, and denial of service.

2.2 LLM Threat Modeling Frameworks

While LLMs are rapidly gaining popularity in industry, security researchers and practitioners are discovering new LLM-specific threats [7]. Frameworks developed by security practitioners, such as OWASP Top 10 for LLM applications¹ and MITRE ATLAS² are essential to create a threat modeling approach for LIAs.

In the same fashion as the classic OWASP Top 10, OWASP Top 10 for LLM shows a collection of the most popular security threats encountered in LIAs, emphasizing their consequences, simplicity of exploitation, and common occurrence in real-world applications. The most frequent LLM threat is Prompt Injection, which represents all attacks where an attacker crafts an input ingested by the target LLM to force the target LLM to misbehave or malfunction [7]. Based on the high value of training data and its impact on the LLM behavior, Training Data Poisoning describes inserting manipulated data points into the training data set of a target LLM [26]. Supply Chain Vulnerabilities cover all other threats targeting an LLM through third-party components [30]. The output of an LLM enables threats, such as Insecure Output (handling), Sensitive Information Disclosure, and Overreliance. Threats referring more to the model operations are called Model Denial of Service and Excessive Agency. Model Denial of Service denotes attacks forcing the LLM to increase its latency and energy consumption [22]. Excessive Agency refers

to the threat that an LLM abuses high access privileges in the system it is embedded in [18]. The last threat on the OWASP Top 10 LLM list is called Model Theft.

MITRE developed and maintains the Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS)³ to organize threats and mitigations related to generic AI systems. ATLAS includes 14 high-level tactics (from reconnaissance to exfiltration) used to group 58 specific attack techniques targeting AI components, six of which are specific to LLMs. Table 1 maps MITRE ATLAS attack techniques to the threats in OWASP’s Top 10 for LLM, including the respective high-level tactics. High-level Tactics describe milestones and rationale for a successful attack⁴. For example, an attacker performs a direct prompt injection to escalate their privileges in the system where the targeted LLM is deployed⁵.

We will utilize OWASP Top 10 for LLMs in our threat modeling approach tailored for LIAs and extend the list of threats collected by MITRE ATLAS.

2.3 Related Work

Researchers addressed LLM-specific threats using a framework consisting of STRIDE and DREAD (Damage, Reproducibility, Exploitability, Affected Users, and Discoverability) following Shostack’s Four Question Framework [23]. Derczynski et al. [3] proposes a method for assessing the risk of deploying an LLM with risk cards. Researchers utilized threat models as guidance for attacks on LIAs in specific contexts. For example, Li et al. [14] describes how a threat model helped to conduct successful attacks on LLMs integrated into the Smart Grid. Security researchers successfully attacked a LIA incorporating ChatGPT3.5 and ChatGPT-4o based on the results of an ad-hoc threat model [12]. As shown in Jiang et al. [12], a threat model can be operationalized for red-team activities and subsequent mitigation actions—such a use case is formalized in a taxonomy presented in [25]. Software engineering security practices applied to LIAs are likely to overlook, mishandle, or ignore Machine Learning (ML) and LLM-specific threats. Jedrzejewski [11] calls for a joint effort between academia and industry. The authors provide a threat modeling method focusing on common issues security practitioners face in the context of ML systems development.

3 Our Vision: the ThreMoLIA Approach

Our vision is to use LLM to support the threat modeling process of LIAs. Figure 1 shows an overview of the LIA threat modeling approach. We propose a threat modeling approach guided by Shostack [21]. As a first step, the LIA is illustrated by a DFD to create an abstraction of its components and their communication. This process leverages relevant and available software engineering artifacts, such as the LIA system design documents and requirements specifications provided by the practitioners. Next, we apply threat frameworks such as MITRE ATLAS, the OWASP Top 10 for LLMs, or STRIDE to identify and assess the most common LLM-specific threats. Thereafter, the approach requires cataloging each LLM-specific threat impacting the LIA using the attack playbooks and tactics described in MITRE ATLAS. Furthermore, we plan to include

¹<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

²<https://atlas.mitre.org/matrices/ATLAS>

³<https://atlas.mitre.org/matrices/ATLAS>

⁴<https://atlas.mitre.org/tactics>

⁵<https://atlas.mitre.org/tactics/AML.TA0012>

MITRE ATLAS	OWASP Top 10 LLM Ranking	High-Level Tactics
(direct and indirect) Prompt Injection	LLM01: Prompt Injection	Initial Access, Persistence, Defense Evasion, Privilege Escalation
LLM Prompt Self-Replication		Persistence
LLM Jailbreak		Defense Evasion, Privilege Escalation
LLM Meta Prompt Extraction	LLM02: Insecure Output Handling	Discovery, Exfiltration
Discover LLM Hallucination		Discovery
LLM Plugin Compromise	LLM07: Insecure Plugin Design	Privilege Escalation, Execution
LLM Data Leakage	LLM06: Sensitive Information Disclosure	Exfiltration

Table 1: Mapping of MITRE Attacks and Tactics to OWASP Top 10 for LLM .

mitigation techniques suggested by MITRE ATLAS to address the identified and assessed threats in the LIA under assessment.

3.1 LLM-based Threat Modeling

The overall goal of ThreMoLIA is to generate threat models with the assistance of LLMs, based on stakeholder prompts and relevant data points in the threat modeling process. Given the non-deterministic character of LLM outputs, the ThreMoLIA approach includes an assessment of the quality of each generated threat model. We designed ThreMoLIA with composability in mind (see Figure 2). In this section, we describe its main components and the foreseen challenges in developing and evaluating them.

Retrieval Augmented Generation (RAG). The RAG component provides the LLM with the necessary resources to reason about a system and generate a threat model. The RAG workflow follows the design summarized by Gao et al. [5], consisting of three stages, indexing, retrieval, and generation. The incoming query represents the part of the stakeholder request (e.g., a prompt) that provides further context to the LLM. The RAG component vectorizes the requested information and compares it against the vectorized documents in the database. Relevant documents are identified based on the closest vectors in the vector space, which represents the vectorized documents from the database. This component forwards the relevant documents to the LLM. The Data Aggregation component fills the database with relevant data from different data sources.

RAG Challenges. One challenge will be vectorizing graphical document types, as this process is more complex than vectorizing purely text-based documents. Another point will be to effectively compare documents to discern whether they are relevant to the threat model of a given LIA without introducing redundancy or useless information.

Data Aggregation. This component collects and aggregates data from different sources, such as natural-language descriptions (e.g., requirement specifications, design documents) and visual representations (e.g., architectural diagrams) of the LIA currently under threat modeling. Another relevant source is the existing threat models collected from previous threat modeling sessions of the same LIA or from similar projects. Moreover, once the LIA is in operation, both pre- and post-release, sensor data collected from the monitoring of its components will enable *continuous* threat

modeling—e.g., when a new component is added to the architecture, the threat model is automatically regenerated and reviewed in the next stakeholder’s threat modeling session.

Data Aggregation Challenges. The selection of data of suitable quality is an issue. Stakeholders conducting threat modeling have varying levels of maturity[31], resulting in heterogeneous data. Moreover, stakeholders apply company-specific terminology, and there exists a variety of DFD styles [19] which can lead to complications when providing context about the LIA. Besides data quality, data needs to be prioritized to meet the context constraints of the LLM powering the ThreMoLIA approach. If too much data is retrieved, important details may be diluted or lost, and, in the worst case, if the content exceeds the model’s token limit, some parts may be cut off, affecting response accuracy. Part of the prioritization is to weigh the data sources, as, for example, LLMs are subject to a source bias, preferring content generated by other LLMs[2].

Prompting. This component constructs a prompt for the LLM powering ThreMoLIA. The basic building block is a predefined system prompt engineered to perform the task of threat modeling. Furthermore, the stakeholders provide a user prompt and a reasoning strategy (e.g., Chain of Thought [32]). Indications of the context supporting the current system under threat modeling, such as the one provided by the existing DFD and its requirements specification, are also fed to the LLM. In practice, this context input will trigger the retrieval step in the RAG component.

Prompting Challenges. We foresee the key challenge in implementing this component to be the development of a prompt template that allows practitioners with varying degrees of security knowledge to interact with ThreMoLIA reliably—i.e., prompts need to be adapted to different stakeholders’ profiles and reflected in the prompting strategy. To that end, we will explore conversation disentanglement techniques [15] to handle multiple stakeholder interactions over (possibly) multiple threat modeling sessions.

Quality Assurance. This component performs several quality assurance steps on the resulting threat model. The first step is to check the syntactical correctness of the output that the stakeholder requested in their prompt (e.g., Open Threat Model⁶).

Further, the output is parsed to extract relevant aspects to validate metamorphic relationships [28], which represent the oracles

⁶<https://github.com/iriusrisk/OpenThreatModel>

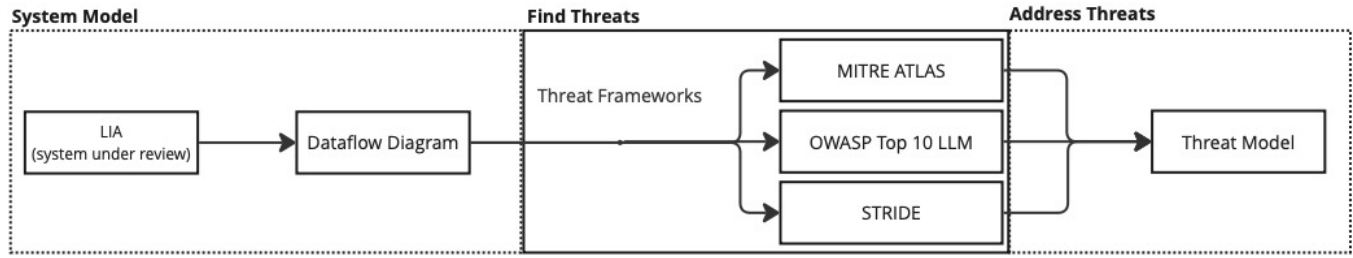


Figure 1: Threat modeling method overview.

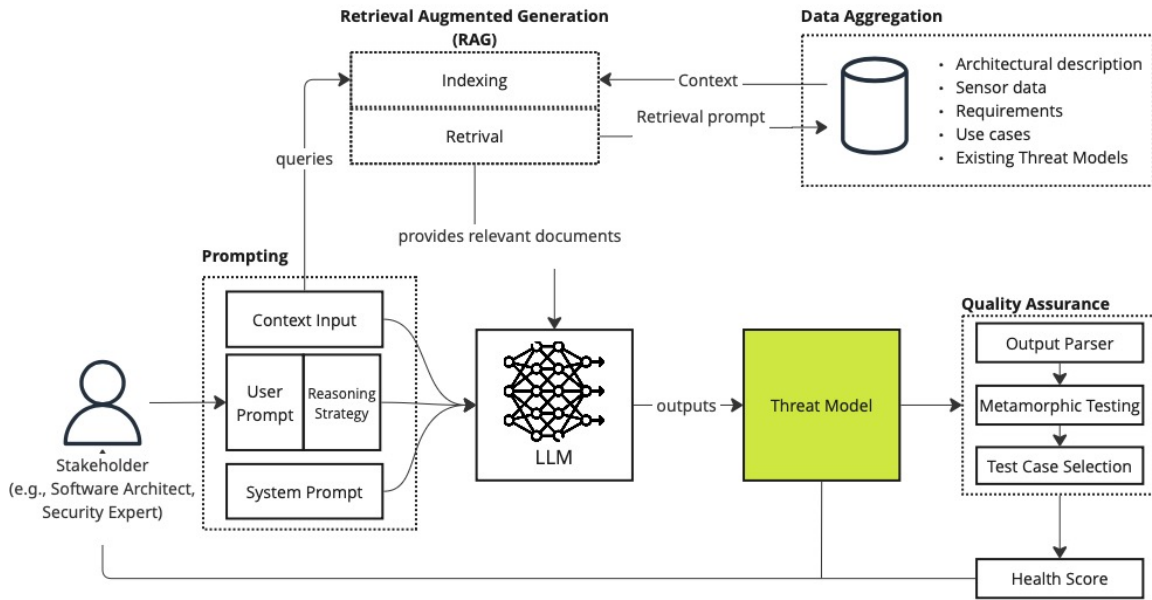


Figure 2: ThreMoLIA overview.

used to execute a test suite. In particular, such a test suite is optimized using different test case selection strategies (e.g., coverage of architectural components). The quality of the generated threat model is summarized in a health score, further communicated to the stakeholders who can decide to further refine their prompt to improve the quality of the threat model.

Quality Assurance Challenges. Currently, established metrics to systematically quantify and evaluate the quality of a threat model are lacking [31]. Those metrics are necessary to derive metamorphic relationships specific to the task of threat modeling as well as to define sensible test case selection strategies. As a starting point, we consider extending the *Machine Learning Security Maturity Model*[10], which proposes a maturity score based on the suggested mitigation techniques applied in a given ML application.

3.2 Evaluation Plan

We aim to evaluate ThreMoLIA and assess the efficiency and effectiveness of the stakeholders using it. There is no agreed-upon benchmark to evaluate a threat model [31]. Our first step is to create such a benchmark by extracting proposed and applied metrics

through a systematic study of the relevant literature. We validate these metrics in focus groups with two security specialists and test their applicability in an industrial case study.

Based on the obtained metrics, we will conduct experiments in industrial settings to compare ThreMoLIA performance with a *traditional* approach—i.e., with the tools currently used by our industrial partner. We plan to conduct a multiple case study as a final assessment of ThreMoLIA once deployed in an industrial environment. The goal is to observe and analyze patterns in multiple products within our industry partner to gain a deeper understanding of how ThreMoLIA fits the workflows of security practitioners.

4 Current state and early results

This section summarizes the preliminary results we collected. The initial prototype reflects our first approach to determine whether applying LLMs during threat modeling is promising. Furthermore, we report a preliminary set of evaluation metrics.

Metric Name	Description
Adversary Capability	Specifies the resources (such as expertise, financial resources, technical resources), methods, and attack vectors.
Attack Success Probability (ASP)	Quantifies the likelihood of a successful attack.
Exposure Level (EL)	Assesses exposed system vulnerabilities for potential attackers.
Impact Severity (IS)	Measures the potential damage or impact of an attack.
Likelihood and feasibility of attacks	Calculates the likelihood of attacks besides the feasibility of attacks (non-software-related case).
Residual Risk (RR)	Calculates risk after security mitigations have been applied.
Identifies phases/elements/patterns in Attacks (composite threats)	Identifies phases, elements, and patterns in attacks, including composite threats.
Accuracy	Represents the number of true positives and true negatives (i.e., correctly identified or omitted threats) produced by the threat modeling tool, divided by the total number of threat classification outcomes.
Threat Coverage (TC)	Indicates the proportion of identified threats addressed by the unified threat model.
Asset Coverage	Percentage of compromised assets in an attack (simulation).
Coverage of MITRE ATT&CK (tactics and techniques)	Represents a new threat modeling approach (coreLang) mapped against the ATT&CK matrix as a form of validation, covering 46%-64% of ATT&CK techniques.
Threat Library	Describes the knowledge sources used for threat identification.
Scalability	Evaluates the ability to scale the model effectively.
Mitigation Effectiveness	Effectiveness of security controls in reducing identified risks.
Threat Model Reusability	Allows reuse of existing threat models when creating new models.
Security Testing	Generates test cases based on the threat model for pentesting.
SDLC Integration	Represents the feasibility of applying the threat modeling tool in the software development lifecycle (SDLC) together with other tools.
Model Complexity (MC)	Assesses the complexity of the unified threat model in terms of the number of nodes and relationships.
Visualization of Risk Models	Provides a visual representation of risk for better comprehension.
Engineer Friendly	Represents user satisfaction with the threat modeling tool.

Table 2: Preliminary Evaluation Metrics.

4.1 Initial Prototype

We conducted a first investigation of how ChatGPT-3.5 Turbo performs in a zero-shot threat modeling task, using the architectural description of a simple LIA. ChatGPT referred to the content of OWASP Top 10 LLM and MITRE ATLAS matrix. In our test, the first prompt contained the request to conduct threat modeling on a system whose architecture we further describe for each component in the same prompt (i.e., without the need to build context from the RAG database). Since we proposed vague threat modeling instructions, ChatGPT gave us the possibility to either create a threat model for the entire system or for a single component without clarifying which threat modeling method, such as STRIDE, would be applied. Furthermore, ChatGPT asked to choose between frameworks, such as OWASP Top 10 LLM and MITRE ATLAS, to follow during the threat modeling task.⁷

4.2 Evaluation Metrics

To evaluate the ThreMoLIA approach, we investigated the literature to extract applied metrics from other studies evaluating threat

modeling approaches. A focus group consisting of three security researchers evaluated the extracted metrics for their applicability and relevance. Table 2 shows a preliminary selection of the metrics, we plan to use to evaluate LIA threat models.

In the next step, two security experts will provide feedback based on the selected metrics and potentially add additional ones not covered by the literature. The metrics will help to evaluate the threat models generated by ThreMoLIA.

5 Conclusion

LLMs can augment the capabilities of software systems and services, but they also introduce a new set of threats that we need to detect and mitigate systematically and reliably. Researchers and practitioners developed and applied threat modeling methods explicitly tailored to deterministic traditional (i.e., non-AI) software. LIAs pose a new challenge for existing threat modeling methods, requiring researchers and practitioners to evaluate their effectiveness and either develop new approaches or adjust existing ones.

In this vision paper, we argue that an approach to support practitioners in threat modeling of LIAs is necessary. To that end, we proposed ThreMoLIA, an LLM-based approach focusing on critical

⁷<https://chat.openai.com/share/7a624c6e-9ec2-4625-9ca3-0f4df8d222cf>

aspects such as context input representation and quality assurance for a complex task such as threat modeling. Moreover, we report the current state of ThreMoLLA development, including an evaluation plan that will be executed in collaboration with an industry partner.

Acknowledgments

We would like to acknowledge that this work was supported by the KKS foundation through the SERT Research Profile project (research profile grant 2018/010) at Blekinge Institute of Technology and the Threat Modeling for LLM-Integrated applications(ThreMoLLA) Research Project supported by Vinnova (Sweden's Innovation Agency) (Diarienummer 2024-00659).

References

- [1] Ricardo Britto, Timothy Murphy, Massimo Iovene, Leif Jonsson, Melike Erol-Kantarci, and Benedek Kovács. 2023. Telecom AI Native Systems in the Age of Generative AI—An Engineering Perspective. *arXiv preprint arXiv:2310.11770* (2023).
- [2] Sunhao Dai, Yuqi Zhou, Liang Pang, Weihao Liu, Xiaolin Hu, Yong Liu, Xiao Zhang, Gang Wang, and Jun Xu. 2024. Neural retrievers are biased towards llm-generated content. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 526–537.
- [3] Leon Derczynski, Hannah Rose Kirk, Vidhisha Balachandran, Sachin Kumar, Yulia Tsvetkov, Mark R Leiser, and Saif Mohammad. 2023. Assessing language model deployment with risk cards. *arXiv preprint arXiv:2303.18190* (2023).
- [4] Yi Dong, Ronghui Mu, Gaojie Jin, Yi Qi, Jinwei Hu, Xingyu Zhao, Jie Meng, Wenjie Ruan, and Xiaowei Huang. 2024. Building guardrails for large language models. *arXiv preprint arXiv:2402.01822* (2024).
- [5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2 (2023).
- [6] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A Smith. 2020. Realltoxicityprompts: Evaluating neural toxic degeneration in language models. *arXiv preprint arXiv:2009.11462* (2020).
- [7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 79–90.
- [8] Kathrin Grosse, Lukas Bieringer, Tarek R Besold, and Alexandre M Alahi. 2024. Towards more Practical Threat Models in Artificial Intelligence Security. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4891–4908.
- [9] Daniela Haluza and David Jungwirth. 2023. Artificial intelligence and ten societal megatrends: an exploratory study using GPT-3. *Systems* 11, 3 (2023), 120.
- [10] Felix Jedrzejewski, Davide Fucci, and Oleksandr Adamov. 2023. MLSMM: Machine Learning Security Maturity Model. *arXiv preprint arXiv:2306.16127* (2023).
- [11] Felix Viktor Jedrzejewski. 2024. Threat Modeling of ML-intensive Systems: Research Proposal. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*. 264–266.
- [12] Fengqing Jiang, Zhangchen Xu, Luyao Niu, Boxin Wang, Jinyuan Jia, Bo Li, and Radha Poovendran. 2023. Identifying and mitigating vulnerabilities in llm-integrated applications. *arXiv preprint arXiv:2311.16153* (2023).
- [13] Anton Konev, Alexander Shelupanov, Mikhail Kataev, Valeriya Ageeva, and Alina Nabieva. 2022. A survey on threat-modeling techniques: protected objects and classification of threats. *Symmetry* 14, 3 (2022), 549.
- [14] Jiangnan Li, Yingyuan Yang, and Jinyuan Sun. 2024. Risks of Practicing Large Language Models in Smart Grid: Threat Modeling and Validation. *arXiv preprint arXiv:2405.06237* (2024).
- [15] Manqing Mao, Paishun Ting, Yijian Xiang, Mingyang Xu, Julia Chen, and Jianzhe Lin. 2024. Multi-user chat assistant (MUCA): a framework using LLMs to facilitate group conversations. *arXiv preprint arXiv:2401.04883* (2024).
- [16] Lara Mauri and Ernesto Damiani. 2022. Modeling threats to AI-ML systems using STRIDE. *Sensors* 22, 17 (2022), 6662.
- [17] Suvda Myagmar, Adam J Lee, and William Yurcik. 2005. Threat modeling as a basis for security requirements. (2005).
- [18] Rahul Pankajakshan, Sumitra Biswal, Yuvaraj Govindarajulu, and Gilad Gressel. 2024. Mapping LLM Security Landscapes: A Comprehensive Stakeholder Risk Assessment Proposal. *arXiv preprint arXiv:2403.13309* (2024).
- [19] Simon Schneider, Nicolas E Diaz Ferreyra, Pierre-Jean Queval, Georg Simhandl, Uwe Zdun, and Riccardo Scandariato. 2024. How Dataflow Diagrams Impact Software Security Analysis: an Empirical Experiment. *arXiv preprint arXiv:2401.04446* (2024).
- [20] Nataliya Shevchenko, Timothy A Chick, Paige O'Riordan, Thomas Patrick Scanlon, and Carol Woody. 2018. Threat modeling: a summary of available methods. *Software Engineering Institute/ Carnegie Mellon University* (2018).
- [21] Adam Shostack. 2014. *Threat modeling: Designing for security*. John Wiley & Sons.
- [22] Ilia Shumailov, Yiren Zhao, Daniel Bates, Nicolas Papernot, Robert Mullins, and Ross Anderson. 2021. Sponge examples: Energy-latency attacks on neural networks. In *2021 IEEE European symposium on security and privacy (EuroS&P)*. IEEE, 212–231.
- [23] Stephen Burabari Tete. 2024. Threat Modelling and Risk Analysis for Large Language Model (LLM)-Powered Applications. *arXiv preprint arXiv:2406.11007* (2024).
- [24] Sven Törpe. 2017. The trouble with security requirements. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*. IEEE, 122–133.
- [25] Apurv Verma, Satyapriya Krishna, Sebastian Gehrmann, Madhavan Seshadri, Anu Pradhan, Tom Ault, Leslie Barrett, David Rabinowitz, John Doucette, and NhatHai Phan. 2024. Operationalizing a threat model for red-teaming large language models (llms). *arXiv preprint arXiv:2407.14937* (2024).
- [26] Alexander Wan, Eric Wallace, Sheng Shen, and Dan Klein. 2023. Poisoning language models during instruction tuning. In *International Conference on Machine Learning*. PMLR, 35413–35425.
- [27] Johannes Welbl, Amelia Glaese, Jonathan Uesato, Sumanth Dathathri, John Mellor, Lisa Anne Hendricks, Kirsty Anderson, Pushmeet Kohli, Ben Coppin, and Po-Sen Huang. 2021. Challenges in detoxifying language models. *arXiv preprint arXiv:2109.07445* (2021).
- [28] Xiaoyuan Xie, Zhiyi Zhang, Tsong Yueh Chen, Yang Liu, Pak-Lok Poon, and Baowen Xu. 2020. METTLE: A metamorphic testing approach to assessing and validating unsupervised machine learning systems. *IEEE Transactions on Reliability* 69, 4 (2020), 1293–1322.
- [29] Wenjun Xiong and Robert Lagerström. 2019. Threat modeling—A systematic literature review. *Computers & security* 84 (2019), 53–69.
- [30] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing* (2024), 100211.
- [31] Koen Yskout, Thomas Heyman, Dimitri Van Landuyt, Laurens Sion, Kim Wuyts, and Wouter Joosen. 2020. Threat modeling: from infancy to maturity. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results*. 9–12.
- [32] Yadong Zhang, Shaoguang Mao, Tao Ge, Xun Wang, Yan Xia, Man Lan, and Furu Wei. 2024. K-Level Reasoning with Large Language Models. *arXiv preprint arXiv:2402.01521* (2024).